Project Blueprint

DocuHelp: Deep Learning-Powered Document Summarizer and Topic Classifier

ITAI 2376 – Deep Learning Artificial Intelligence
Midterm Project – AI Agent Blueprint
Jaret Sanchez

Problem Statement

As students, we encounter long documents in almost all our classes that are time-consuming to read and are difficult to summarize quickly. We have to read research papers, technical reports, lecture notes, articles, and even entire books. This creates a big issue when you are a fulltime student, and have other responsibilities such as a fulltime job, and a family that you need to take care of. That's the reason I want to create DocuHelp, a tool that can help users to determine core ideas, major themes, and essential takeaways. This problem affects not only students, but also researchers, employees, and anyone who has to work with huge amounts of data/documents.

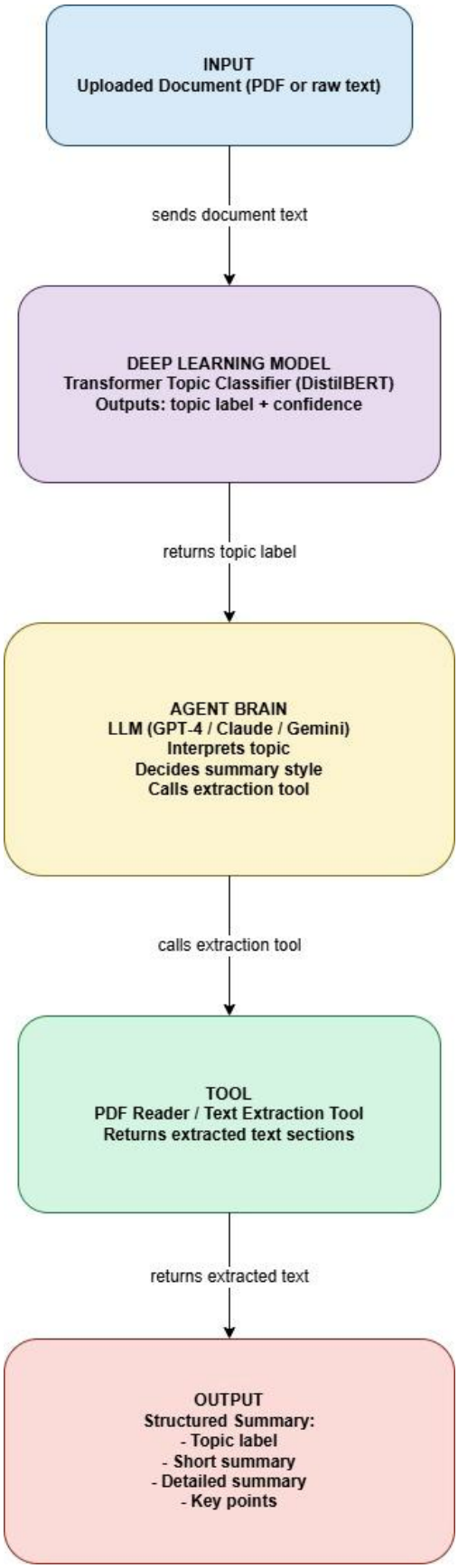
DocuHelp will act as an intelligent document-understanding agent and help us extract the essential information. The agent will accept a PDF or text document, and identify the main topics with the use of a deep learning classifier, and generate a document summary for the user. The main goal of DocuHelp is to help students identify the main subjects and summarize long documents so they are able to understand complex topics and reduce information overload.

Option Choice

For my project, I decided to go with option A and use a single AI agent. I think that for the problem I'm trying to solve, only one agent will be necessary to classify the document's topic, extract the main ideas, and generate a summary. If I had used additional agents, it would have overcomplicated the whole project. A single agent design that uses deep learning will be able to perceive, reason, and act on this real-world issue.

Agent Architecture and Diagram

DocuHelp follows the structure: Input → Deep Learning Model → Agent Brain → Tool → Output. The Agent uses a deep learning classifier to identify the document's topic, an LLM as the reasoning engine, and a PDF-reading tool to extract text. We can observe the single-agent architecture in the diagram below. It shows the flow of information from user input all the way to the final summary output.


INPUT
Uploaded Document (PDF or raw text)
sends document text
DEEP LEARNING MODEL
Transformer Topic Classifier (DistilBERT)
Outputs: topic label + confidence
returns topic label
AGENT BRAIN
LLM (GPT-4 / Claude / Gemini)
Interprets topic
Decides summary style
Calls extraction tool
calls extraction tool
TOOL
PDF Reader / Text Extraction Tool
Returns extracted text sections
returns extracted text
OUTPUT
Structured Summary:
- Topic label
- Short summary
- Detailed summary
- Key points

Deep Learning Connection

DocuHelp integrates two deep learning modules from the course, Sequence Modeling and NLP, and Transformers and Attention Mechanism. These two modules directly support the agent's ability to determine the document's topics and generate the summary.

Sequence Modeling and NLP

The first deep learning component is a text classification model that will be built using sequence-based NLP techniques. In this module, we covered how neural networks process text as sequences of tokens. This type of model is able to capture semantic meaning, sentence structure, and contextual relationships. In my project, I will use what I learned in this module to process the document text, identify the main topics, and provide a structured signal to the LLM brain, allowing me to ensure the agent understands the document before it generates the output.

Transformers and Attention Mechanisms

The second deep learning component is the transformer architecture, which is the one that powers the classifier and underlies the LLM brain. In this module, I learned about self-attention, multi-head attention, and modeling long-range dependencies in text. The use of transformers in my project is ideal because they understand semantic meaning, capture relationships between words, produce embeddings for classification, and enable LLM reasoning.

Agent Framework

I plan to use LangGraph (a graph-based extension of LangChain) to build my project. LangGraph is a great option for my project because it provides clear sequential steps, built-in support, state management for multi-step reasoning, and it easily integrates with deep learning models. I did consider using CrewAI, but LangChain is better suited for a single-agent design.

Tools and Data

The tools DocuHelp will use are:

1. PDF reader/text extraction tool – This tool is needed to extract text from the uploaded documents, and will allow the agent to process PDFs.
2. Text splitter – This tool will break long documents into smaller sections and will help me ensure that the output summary is accurate.
3. Vector Database – This tool will be used to store document chunks for retrieval augmentation, and will help the agent handle long documents.

The agent will work with user-uploaded documents. To train the model, I will use AG News and 20 Newsgroups. These are free datasets that are widely used and appropriate for fine-tuning the model. Both datasets are available on Kaggle.

Build Plan

WEEK	PLAN
Week 1 (April 6-13)	• Set up the environment • Implementing PDF loading and text extractor • Define categories and data schema
Week 2 (April 14-20)	• Fine-tune the transformer classifier • Test classification on sample documents • Integrate the classifier into the pipeline
Week 3 (April 21-27)	• Build the full agent loop • Implement summary generation • Test and refine prompts
Week 4 (April 28-May 3)	• Polish the agent's reasoning • Write final report • Submit all the required files

Anticipated Challenges

Some of the challenges that I can encounter include:

- Long documents exceeding LLM context limits – Some documents can be too long for the LLM to process in a single pass. To handle this issue, I would use text splitting and retrieval-augmentation.
- Topic classifier mislabeling ambiguous documents – Documents with overlapping topics can confuse the classifier. To handle this issue, I would use confidence thresholds and fallback prompts.
- PDF extraction noise – PDFs can contain formatting issues. To handle this issue, I would use preprocessing steps and test different documents to ensure a clean text extraction.

References

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). *Attention is all you need*. Advances in Neural Information Processing Systems.

Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). *DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter*. arXiv preprint arXiv:1910.01108.

LangChain. (2023). *LangChain documentation*. https://python.langchain.com LangGraph. (2024). *LangGraph documentation*. https://langchain-ai.github.io/langgraph/ (langchain-ai.github.io in Bing).